

TDCommons D3

Biometric Birth Ephemeris as Deterministic Parameters for Shamir Secret Sharing Key Recovery

Authors: Frédéric Renault

Institution: G1FabLab / UPlanet ORIGIN

License: Creative Commons Attribution 4.0

Code: <https://github.com/papiche/Astroport.ONE> (AGPL-3.0)

Abstract

This disclosure describes a method for deriving cryptographic identity keys and Shamir Secret Sharing (SSSS) parameters deterministically from biometric birth and conception ephemeris data, eliminating central biometric storage.

Two distinct canonical ASCII strings serve as key derivation inputs:

SALT (birth anchor): YYYYMMDDHHMM_LAT_LON_POLARITY_WEIGHT

Example: 198504171530_48.86_2.35_0_3.2

PEPPER (conception anchor): YYYYMMDDHHMM_LAT_LON_WEIGHT

Example: 198407110830_48.86_2.35_3.2

If the exact conception timestamp is unknown, it is algorithmically deduced from the birth timestamp using a weight-modulated gestation formula: $\text{gestation_days} = 280.0 + (\text{birth_weight_kg} - 3.5) \times 4.0$.

The SALT and PEPPER, combined with an email identifier, are fed into a Key Derivation Function to produce a 32-byte seed compatible with BIP-39 / secp256k1. The seed is split using Shamir's (2,3) threshold scheme, distributed across the user's device, a local service node, and a cooperative network node.

This approach allows full cryptographic key recovery on any device solely from immutable birth parameters, without biometric hardware (fingerprint readers, iris scanners) or centralized government ID databases.

1. Background

Existing cryptographic identity recovery systems rely on: - Hardware security modules (biometric readers) - Centralized key escrow databases - Paper backup of random seed phrases (user-managed, loss-prone) - Government-issued document verification (slow, centralized)

This disclosure presents a deterministic approach: the same human birth and conception circumstances always yield the same cryptographic key material, enabling recovery without any of the above infrastructure.

2. SALT Derivation (Birth Data)

2.1 Input Parameters

Parameter	Source	Precision	Notes
birth_year	Birth certificate	YYYY	4 digits
birth_month	Birth certificate	MM	2 digits, zero-padded
birth_day	Birth certificate	DD	2 digits, zero-padded
birth_hour	Birth certificate	HH	Local time, 2 digits
birth_min	Birth certificate	MM	2 digits, zero-padded
birth_lat	Birth location	0.01°	snappedf(lat, 0.01), format %.2f
birth_lon	Birth location	0.01°	snappedf(lon, 0.01), format %.2f
polarity	Self-declared	binary	0 = Φ -wave, 1 = Octave-wave
birth_weight_kg	Birth certificate	0.1 kg	format %.1f

2.2 SALT String Format

GDScript format string: "%04d%02d%02d%02d%02d_%.2f_%.2f_%d_%.1f"

Fields in order: year, month, day, hour, minute, latitude, longitude, polarity, weight

Example:

Birth: 17 April 1985, 15:30 local, Paris (48.86°N, 2.35°E), polarity 0, weight 3.2 kg
SALT = "198504171530_48.86_2.35_0_3.2"

2.3 Design Rationale

- **0.01° precision** (~1 km resolution): provides enough geographic specificity to distinguish birth locations while tolerating minor inaccuracies in transcription
 - **Local time** (not UTC): birth certificates record local time; the UTC conversion is not always reliably known
 - **Polarity** (binary, not text): avoids encoding ambiguity, language-independent
 - **Weight to 1 decimal** (%.1f): distinguishes 3.2 kg from 3.3 kg without false precision; the format %.1f always produces exactly one decimal digit (e.g., 3.2, not 3.20)
-

3. PEPPER Derivation (Conception Data)

3.1 Gestation Duration Computation

If the exact conception date/time is not known by the user, it is estimated algorithmically from the birth timestamp and birth weight using the following formula:

$$\text{gestation_days} = 280.0 + (\text{birth_weight_kg} - 3.5) \times 4.0$$

Derivation: - 280 days = standard human gestation (40 weeks from last menstrual period) - The weight offset $(\text{weight} - 3.5) \times 4.0$ models the empirical correlation between birth weight and gestation duration (heavier babies tend to gestate longer) - A 3.5 kg reference weight is used as the neutral point (population mean) - Scale factor: 4.0 days per 0.1 kg deviation

Example:

```
birth_weight = 3.2 kg
gestation_days = 280.0 + (3.2 - 3.5) × 4.0 = 280.0 - 1.2 = 278.8 days
conception_unix = birth_unix - round(278.8 × 86400)
```

This formula is **only applied when the user has not manually entered the conception date**. When the exact conception date is known (user-entered), that date/time is used directly as `conception_unix`.

3.2 Input Parameters

Parameter	Source	Precision	Notes
conception_unix	User-entered or computed	seconds	UTC Unix timestamp
con_year	Extracted from conception_unix	YYYY	UTC year
con_month	Extracted from conception_unix	MM	UTC month
con_day	Extracted from conception_unix	DD	UTC day

Parameter	Source	Precision	Notes
con_hour	Extracted from conception_unix	HH	UTC hour (not local)
con_min	Extracted from conception_unix	MM	UTC minute
con_lat	Conception location	0.01°	snappedf(lat, 0.01), format %.2f
con_lon	Conception location	0.01°	snappedf(lon, 0.01), format %.2f
birth_weight_kg	Same as SALT	0.1 kg	format %.1f

Important: The UTC offset (conception_utc_offset_h) is used exclusively for geographic Goldberg polyhedron anchor computation in the user interface. It is **not included** in the PEPPER string. The PEPPER relies strictly on UTC-derived date/time components extracted directly from conception_unix.

3.3 PEPPER String Format

GDScript format string: "%04d%02d%02d%02d%02d_%.2f_%.2f_%.1f"

Fields in order: UTC year, UTC month, UTC day, UTC hour, UTC minute, latitude, longitude, weight

Example:

Conception: 11 July 1984, 08:30 UTC, Paris (48.86°N, 2.35°E), birth weight 3.2 kg

PEPPER = "198407110830_48.86_2.35_3.2"

Note: when the gestation formula is used, the conception location defaults to the birth location (same lat/lon) unless the user provides a different conception location.

4. Key Derivation

4.1 Full Derivation Chain

INPUT:

```

SALT    = "YYYYMMDDHHMM_LAT_LON_POLARITY_WEIGHT"
PEPPER  = "YYYYMMDDHHMM_LAT_LON_WEIGHT"
EMAIL   = user's email address (identifier, not stored on-chain)

```

STEP 1 – Primary key (nsec1):

```

seed1 = KDF(password=SALT, salt=EMAIL, iterations=100000)
nsec1 = secp256k1_privkey_from_bytes(seed1[:32])

```

STEP 2 – Secondary key (nsec2):

```
seed2 = KDF(password=PEPPER, salt=nsec1_hex, iterations=100000)
nsec2 = secp256k1_privkey_from_bytes(seed2[:32])
```

Where KDF is PBKDF2-HMAC-SHA512, Argon2id, or Scrypt. The actual KDF is handled by the Astroport node at key creation time. The node receives only the formatted SALT and PEPPER strings and returns the derived public key (npub); the private key (nsec) is computed locally on-device or reconstructed via SSSS.

4.2 Key Types

Key	Derived from	Purpose
nsec1	SALT + EMAIL	Primary NOSTR identity (published on relay)
nsec2	PEPPER + nsec1	Secondary identity (private channels, ZEN economy)
npub	from nsec1	Public key, published on NOSTR

5. SSSS Key Recovery

5.1 Share Distribution

The 32-byte seed from Step 1 is split using Shamir’s Secret Sharing with threshold (2, 3):

Share	Holder	Storage
Share 1	User	Device local storage (encrypted)
Share 2	Capitaine	Local trusted service node
Share 3	Armateur	Cooperative network hub node

Any 2 of 3 shares reconstruct the original seed. No single party holds sufficient information for reconstruction alone.

5.2 Recovery Scenario

If the user loses their device: 1. User contacts their Capitaine (local node operator) 2. User re-enters birth ephemeris data → SALT recomputed 3. Capitaine provides Share 2 4. Armateur (network hub) provides Share 3 5. Shares 2 + 3 reconstruct the seed, verifying against the SALT-derived public key 6. nsec1 restored without any central database lookup

5.3 Security Properties

Separation of factors: - “Something you are/know” = birth ephemeris (SALT) — in the user’s memory - “Something you have” = device (Share 1) + network nodes (Shares 2, 3)

No biometric hardware required: The system does not use fingerprint readers, iris scanners, or face recognition. The “biometric” factor is the user’s knowledge of their birth data.

No central database: No entity holds the full SALT+PEPPER combination. The network nodes hold only SSSS fragments.

PEPPER as entropy amplifier: The PEPPER adds a second derivation layer that an attacker cannot reconstruct without knowledge of the conception date and location, which are typically unknown to anyone except the user and close family.

6. Comparison with Existing Systems

Feature	This System	Hardware Biometrics	Paper Seed	Central Escrow
Recovery without device	✓ (via SSSS)	✗ (hardware required)	✓	✓
No central database	✓	✗	✓	✗
No specialized hardware	✓	✗	✓	✓
Immune to loss/forgetting	✓ (birth data = unforgettable)	✗	✗ (seed loss)	✓
Immune to coercion	Partial	✗	Partial	✗
Deterministic reproducibility	✓	Hardware-dependent	✓	✓

7. Prior Art and Novelty

Known prior art: - Shamir Secret Sharing: Shamir (1979) - PBKDF2: RFC 2898 (2000) - BIP-39 mnemonic key derivation: Bitcoin Improvement Proposal (2013) - Memory-hard KDFs (Argon2id): RFC 9106 (2021)

Novel combination: No prior art is known for: 1. Using birth date/time/location/weight as the SALT string for deterministic secp256k1 key derivation 2. Using algorithmically-inferred conception date (via $280 + (\text{weight} - 3.5) \times 4.0$ gestation formula) as the PEPPER string 3. The specific ASCII format strings YYYYMMDDHHMM_LAT_LON_POLARITY_WEIGHT and YYYYMMDDHHMM_LAT_LON_WEIGHT for canonical key material encoding 4. Combining (1)-(3) with Shamir (2,3) distributed across user device + Capitaine + Armateur roles

8. Claims of Novelty

1. A SALT string format YYYYMMDDHHMM_LAT_LON_POLARITY_WEIGHT derived from birth ephemeris data with 0.01° geographic precision and 0.1 kg weight precision for deterministic cryptographic key derivation.
2. A PEPPER string acting as a **cryptographic entropy stretch function**: the formula $\text{gestation_days} = 280.0 + (\text{birth_weight_kg} - 3.5) \times 4.0$ deterministically derives the conception Unix timestamp from the birth timestamp and birth mass, linking a physically measurable and unpredictable quantity (birth weight at 0.1 kg resolution) to a temporal offset, multiplying the key space without requiring any additional user input.
3. The two-layer key derivation: nsec1 from SALT+EMAIL, nsec2 from PEPPER+nsec1. The two layers are linked (nsec2 uses nsec1 as its KDF salt), ensuring that a correct PEPPER alone is insufficient to derive nsec2 without also knowing nsec1.
4. A deterministic key derivation framework devoid of centralized biometric templates, relying solely on the immutability of historical localized human ephemeris (birth time, birth location, birth mass, conception time, conception location) acting as an **un-extractable brain-wallet seed**: the credentials exist only in human long-term memory and cannot be extracted by hardware attacks, rubber-hose cryptanalysis targeting physical devices, or central database breaches.

This framework supports **Bounded Fuzzy Derivation**: because the human error space is tightly constrained (e.g., ± 0.2 kg on birth weight $\rightarrow$ 5 candidate values; ± 30 minutes on birth time $\rightarrow$ 13 candidate values; their cartesian product $\approx$ 200–500 candidate SALT strings), a client application can locally enumerate all plausible variants and test each against the user's existing public NOSTR profile (which contains the derived public key npub). The candidate whose derived public key matches the stored npub is the correct SALT, recovering the private key without weakening cryptographic entropy: an external attacker who knows nothing about the individual faces the full key space (2^{256}), while the legitimate user faces only $O(500)$ local computations. No network request is required for this recovery.

5. The specific SSSS (2,3) distribution model mapping shares to User device / Capitaine (local trusted operator) / Armateur (cooperative hub) roles, where no single cooperative entity can reconstruct the seed unilaterally.

This disclosure is submitted for defensive publication purposes only. The authors do not seek patent protection for the described methods and explicitly place this disclosure in the public domain of prior art.